Федеральное агентство по рыболовству

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Астраханский государственный технический университет»

Система менеджмента качества в области образования, воспитания, науки и инноваций

ООО «ДКС РУС» по международному стандарту ISO 9001:2015

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОММУНИКАЦИЙ

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЗАЩИТА ИНФОРМАЦИИ

ОТЧЕТ

о выполнении лабораторной работы № 5

Комплексное обеспечение безопасности менеджера учётных записей

Выполнил:
студент гр. ЗИНРб-51

_____ Генералов В. В.

«_____» _____ 2026 г.

Преподаватель:

_____ Савоськин И. А.

«_____» _____ 2026 г.

Астрахань – 2026

ВВЕДЕНИЕ

Разработан менеджер учётных записей с комплексной защитой базы данных (БД).

Цель работы: «ознакомиться с принципами комплексной защиты информации на примере обеспечения защиты менеджера учетных записей».

Задание по методическим указаниям состоит из четырёх пунктов:

1. «Разработать программу, реализующую управление произвольными учетными записями. Программа должна иметь графический интерфейс, позволяющий просматривать, создавать, удалять и модифицировать имеющиеся учетные записи в файле БД. Пары имя пользователя + пароль должны храниться в отдельном текстовом файле, представляющем собой БД учетных записей. Хранение паролей должно осуществляться в хэшированном виде, т.е. при изменении/добавлении пароля в файл БД записывается его хэш-сумма (пароли не хранятся в открытом виде). Разрешено использовать сторонние функции и процедуры, реализующие алгоритм хэш-функции».
2. «Добавить обеспечение целостности БД учетных записей путем проверки хэш-суммы. При каждом изменении БД посредством программы, в отдельный текстовый файл должна сохраняться хэш-сумма данных из БД. По запросу пользователя программа должна сверять актуальную хэш-сумму БД с эталонной, сохраненной в файле. В случае совпадения программа должна выдавать сообщение, что БД в целостности, при несовпадении – что БД была изменена извне».
3. «Обеспечить криптографическую защиту файлов БД, журнала событий и хэш-суммы при помощи симметричной криптосистемы. Использовать отдельный ключ для каждого файла. Программа должна шифровать/расшифровывать файлы по желанию пользователя. Подсистема шифрования не должна конфликтовать с подсистемой обеспечения целостности. Разрешено использовать сторонние функции и процедуры, реализующие алгоритмы симметричного шифрования».
4. «Добавить в программу журнал событий, представляющий собой отдельный текстовый файл, в который будут заноситься дата и время следующих операций:».

- «Добавление, изменение, удаление учетных записей»;
- «Шифрование/расшифрование файлов»;
- «Сохранение/сравнение хэш-суммы файла БД».

Выполнение задания подтверждают снимки окон, файлы данных и автотест.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Хранение паролей в открытом виде недопустимо: при компрометации БД злоумышленник получает прямой доступ к системе. Вместо пароля хранится его хэш; при проверке вычисляется хэш введённого пароля и сравнивается с сохранённым. Соль – случайная строка, добавляемая к паролю перед хэшированием: одинаковые пароли дают разные хэши, а заранее вычисленные (радужные) таблицы становятся бесполезными. Медленные функции формирования ключа из пароля – PBKDF2 (Password-Based Key Derivation Function 2), bcrypt, Argon2 – повторяют хэширование тысячи раз и во столько же раз удорожают перебор.

Целостность данных также контролируется хэш-функцией, например алгоритмом SHA-256 (Secure Hash Algorithm с 256-битным результатом): при сохранении вычисляется хэш-сумма, которая хранится отдельно; позднее она вычисляется заново и сравнивается с эталонной. Любое изменение данных меняет хэш, а подобрать иные данные с тем же хэшем практически невозможно, поэтому совпадение означает неизменность.

Конфиденциальность файлов обеспечивает симметричное шифрование: один ключ служит и для шифрования, и для расшифрования. Стандарт шифрования AES (Advanced Encryption Standard, федеральный стандарт США FIPS-197) шифрует блоки по 16 байт, выполняя от десяти до четырнадцати раундов подстановок и перестановок: AES-256 после начального AddRoundKey выполняет четырнадцать раундов SubBytes, ShiftRows, MixColumns (в последнем раунде опускается), AddRoundKey; S-блок вычисляется из арифметики поля Галуа $GF(2^8)$. Режим сцепления блоков шифртекста CBC (Cipher Block Chaining) связывает блоки через сложение по модулю 2 с предыдущим блоком шифртекста и случайный вектор инициализации (IV): $\boldsymbol{C_i} = \boldsymbol{E}(\boldsymbol{D_i} \oplus \boldsymbol{C_{i-1}})$, $\boldsymbol{C_0} = \mathrm{IV}$, где $\boldsymbol{D_i}$ и $\boldsymbol{C_i}$ – блоки данных и шифртекста, $\boldsymbol{E}$ – зашифрование блока. Поэтому одинаковые открытые тексты дают разные шифртексты. Перед шифрованием данные дополняются до длины, кратной 16 байтам, по схеме PKCS#7 (Public-Key Cryptography Standards #7). Ключ выводится из пароля функцией PBKDF2 со случайной солью, а код аутентификации сообщения на основе хэш-функции HMAC (Hash-based Message Authentication Code) подтверждает, что шифртекст не изменён и ключ верен.

Подсистема аудита фиксирует дату, время и содержание значимых действий, что позволяет выявлять подозрительную активность и расследовать инциденты.

2 ОПИСАНИЕ АЛГОРИТМА И ПРОГРАММЫ

2.1 Постановка задачи

Требования задания приведены во введении, алгоритмы и их параметры с обоснованием – в таблице 2.1. Где задание не задаёт конкретных средств, приняты следующие решения:

1. Функции PBKDF2-HMAC-SHA256, SHA-256 и HMAC-SHA256 взяты из модулей ***hashlib*** и ***hmac*** стандартной библиотеки Python, а шифр AES-256 в режиме CBC реализован самостоятельно по FIPS-197 (сторонние реализации разрешены, но не требуются) и проверен на векторах стандарта.
2. Строка БД – `логин:соль:хэш`, соль и хэш шестнадцатеричные; эталон обновляется при каждом изменении БД через программу и кнопкой «Сохранить хэш-сумму».
3. Хэш-сумма считается по открытому содержимому БД, шифрование обратимо байт в байт; пока файл зашифрован, операции с ним недоступны, а при зашифрованном журнале доступно только его расшифрование – так подсистемы не конфликтуют.
4. В журнал попадают также проверки паролей и отказы при неверном пароле файла БД или хэш-суммы (отказ по самому журналу записать некуда – он зашифрован).
5. Если файл БД удалён извне, а эталон остался, БД не создаётся заново молча: проверка сообщает об удалении, пустую БД создаёт кнопка «Добавить».
6. БД и эталон меняются только вместе, а операция без записи в журнале не выполняется: если эталон или строку журнала записать не удалось, изменённые файлы возвращаются к прежнему содержимому.

Таблица 2.1 – Параметры реализации

Параметр	Значение	Обоснование
Хэш пароля	PBKDF2-HMAC-SHA256, 100 000 итераций, соль 16 байт	Разные хэши одинаковых паролей; перебор замедлен
Хэш-сумма БД	SHA-256 файла БД, в эталоне записывается вместе с именем файла	Стандартная функция, лавинный эффект
Шифр	AES-256, режим CBC, дополнение PKCS#7, случайный IV	FIPS-197; одинаковые файлы дают разные шифртексты
Ключ файла	PBKDF2 (пароль, соль ‖ имя файла), 200 000 итераций, 64 байта	Отдельный ключ для каждого файла; разделение ключей шифра и HMAC
Аутентификация	HMAC-SHA256 по соли, IV и шифртексту	Неверный пароль и повреждение обнаруживаются до расшифрования

Имя файла входит в соль ключа: ключи файлов различны даже при одинаковых паролях.

2.2 Описание алгоритма

Пароли. Для логина L и пароля P с солью s из 16 случайных байт вычисляется хэш H:

$$H = \text{PBKDF2}(\text{HMAC-SHA256}, P, s, 100\,000). \quad (2.1)$$

В БД записывается строка $L{:}s{:}H$. При проверке введённый пароль P' хэшируется по формуле (2.1) с сохранённой солью s, и результат сравнивается с H за постоянное время.

Целостность. Эталонная хэш-сумма (эталон) $h = \text{SHA-256}(F)$, где F – содержимое файла БД учётных записей, записывается в отдельный файл после каждого изменения БД. По запросу вычисляется актуальная хэш-сумма h' и сравнивается с h: при равенстве выводится сообщение «БД в целостности», иначе – «БД была изменена извне».

Шифрование файла. Из пароля файла K, случайной соли файла s_f (16 байт) и имени файла N по формуле (2.2) выводятся 64 байта, которые делятся на ключ шифрования K_{enc} и ключ HMAC K_{mac} по 32 байта. Содержимое файла D, дополненное по схеме PKCS#7, шифруется AES-256-CBC со случайным вектором инициализации IV в шифртекст C, к которому добавляется код аутентификации T:

$$(K_{\text{enc}}, K_{\text{mac}}) = \text{PBKDF2}(\text{HMAC-SHA256}, K, s_f \parallel N, 200\,000), \quad (2.2)$$

$$C = \text{AES-256-CBC}_{K_{\text{enc}}}(\text{IV}, D),\ T = \text{HMAC-SHA256}_{K_{\text{mac}}}(s_f \parallel \text{IV} \parallel C).$$

Файл заменяется текстом: строка-признак и base64($s_f \parallel$ IV $\parallel C \parallel T$). При расшифровании сначала пересчитывается T: несовпадение означает неверный пароль или повреждение файла, и расшифрование не выполняется.

2.3 Описание программы

Программа написана на Python 3 с использованием только стандартной библиотеки и состоит из трёх модулей: модуль окна реализует графический интерфейс на tkinter, модуль хранилища содержит класс ***Store*** с логикой всех подсистем защиты и автотест, модуль шифрования AES реализует шифр в режиме CBC и его самопроверку на векторах FIPS-197. Программа запускается из консоли и открывает главное окно; автотест хранилища и самопроверку шифра можно выполнить отдельно, без графического интерфейса.

Класс ***Store*** работает с тремя файлами в папке данных рядом с программой: файлом БД учётных записей, файлом эталонной хэш-суммы и файлом журнала событий. Методы ***add_account***, ***update_account***, ***delete_account***, ***verify_password*** ведут учётные записи (логин без пробелов, двоеточий и ведущего «#», пароль от 1 до 128 символов, дубликаты отклоняются); ***save_hash*** и ***check_integrity*** – целостность; ***encrypt_file*** и ***decrypt_file*** – шифрование по паролю файла; ***log*** – журнал.

Главное окно программы содержит четыре блока: «Учётные записи» – таблица (логин,

соль, хэш) с кнопками «Добавить», «Изменить» (переименование и/или новый пароль), «Удалить», «Проверить пароль», «Обновить»; «Шифрование файлов» – состояние каждого файла (например, «открыт, 411 байт» или «ЗАШИФРОВАН») и кнопки «Зашифровать» (пароль вводится дважды) и «Расшифровать»; «Целостность БД» – эталонная и актуальная хэш-суммы, кнопки «Сохранить хэш-сумму» и «Проверить целостность»; «Журнал событий» – содержимое файла журнала. Строка состояния внизу дублирует результат последней операции.

Вид главного окна приведён на рисунке 2.1.



Рисунок 2.1 – Главное окно программы

Кнопки «Добавить» и «Изменить» открывают диалог учётной записи, показанный на рисунке 2.2: пароль вводится дважды, при изменении пустое поле пароля оставляет прежний пароль.

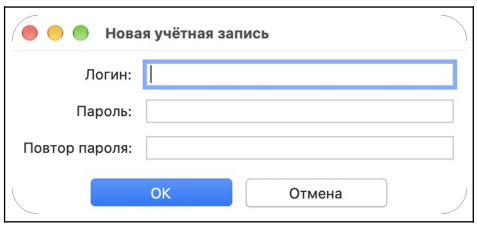


Рисунок 2.2 – Диалог добавления учётной записи

Изменение, удаление и проверка пароля относятся к выделенной строке, пароль файла вводится в диалоге «Ключ файла»; ошибки пользователя (зашифрованный или повреждённый файл, неверный пароль, дубликат, нет прав на запись) показываются диалоговым окном, причём условия операции проверяются ещё до открытия диалога ввода.

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.1 Учётные записи

Снимки окон и содержимое файлов получены в нескольких запусках программы с заново созданной БД, поэтому соли и хэш-суммы на разных рисунках могут различаться. В БД, показанной в окне программы (см. рис. 2.1), три учётные записи, пароль одной из них изменён; таблица показывает логин, соль и хэш – пароли не отображаются и не хранятся. Файлы БД и эталона приведены на рисунке 3.1: у каждой записи своя соль, паролей `qwerty123`, `NewPass#7`, `S3cret!` нет.

```
Файл БД учётных записей:
# логин:соль(hex):PBKDF2-HMAC-SHA256(пароль, соль, 100000 итераций)(hex)
ivan:4ed20390db88640df1f0995c30372c27:
     575e1d8c6c585d6d193f5cf60357d3aa77d37976a9318333b863497bdfc129e6
maria:62d4c3285bee3ec40d982530ab512712:
     691d7c473693bac2f677eb122032573063411ac7aae861451eb0e5b321fec31d
admin:db0f5d6d1ddefc632fd54420b27acdce:
     199fc3ab4b1a8f58e7172c18b5c88b52906ec33a1c27445cda0ca14cfe8dbf6a

Файл эталонной хэш-суммы:
b931004efa1b0536a427b693eded4a2d705635f38d725bda6aad77b170758274  accounts.txt
```

Рисунок 3.1 – Содержимое файлов БД учётных записей и эталонной хэш-суммы

Утечка такого файла не даёт входа в систему, а одинаковые пароли разных пользователей имеют разные хэши. Проверка пароля занимает около 12 мс (100 000 итераций PBKDF2) – примерно 85 паролей в секунду против 2,7 млн вызовов SHA-256 в секунду: словарь из миллиона паролей перебирается за 3,3 часа вместо доли секунды.

3.2 Контроль целостности

Для нетронутой БД проверка сообщает «БД в целостности», как показано на рисунке 3.2 слева. После дописывания строки `hacker:…` в файл БД текстовым редактором проверка сообщает «БД была изменена извне» и выводит эталонную и актуальную хэш-суммы (рисунок 3.2, справа; части рисунка получены в разных запусках).

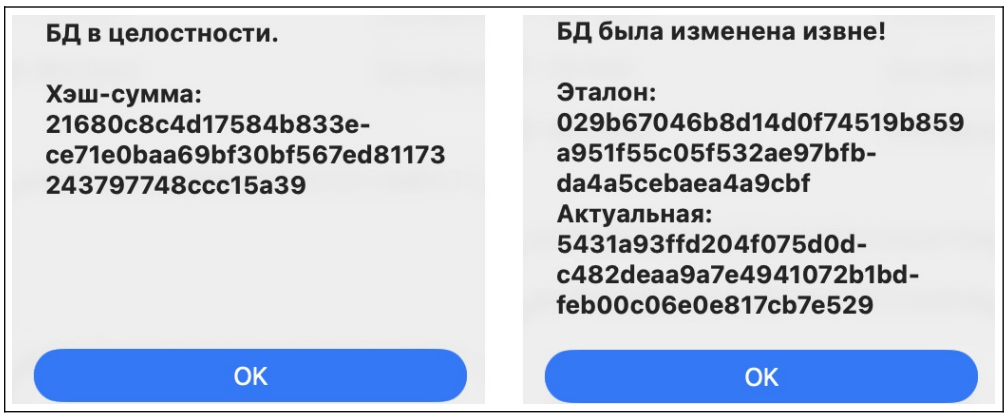


Рисунок 3.2 – Проверка целостности: слева – БД не изменена, справа – после правки файла БД извне

Дописанная извне строка изменила около половины бит хэш-суммы – лавинный эффект

SHA-256; подобрать правку с тем же хэшем практически невозможно, поэтому обнаруживается любое изменение файла. Ложных срабатываний нет: эталон обновляется вместе с БД при каждом изменении через программу, а удаление записи через программу или кнопка «Сохранить хэш-сумму» снова согласуют БД и эталон.

3.3 Шифрование файлов

Кнопка «Зашифровать» запрашивает пароль файла с повтором. После шифрования БД и эталона их состояние – «ЗАШИФРОВАН», список учётных записей – «(БД зашифрована)», поля хэш-сумм – «(файл зашифрован)», как видно на рисунке 3.3; операции с этими файлами блокируются (таблица 3.1).



Рисунок 3.3 – БД и файл хэш-суммы зашифрованы

Вид зашифрованного файла эталонной хэш-суммы показан на рисунке 3.4.

```
Файл эталонной хэш-суммы после шифрования:
#ENCRYPTED AES-256-CBC, HMAC-SHA256, ключ: PBKDF2-HMAC-SHA256 из пароля файла
ZuxFN9YmEftp8dngmFn0r5zPoei8ksNqiNbY99ALciKwyGG05223XEoQIMLoq3GVG/7DUOheYs29
hPAbenQWvI33gZlZUMGSPyUw+EiR9I5MAf04JT5vA7bmspkZjPmPiobiNkDqV2LM4c454Znen2eJ
Gc0lRZwfJEJZHh8T7lGK/3PcQzIqHjzgVVKgIOv8

Расшифрование с неверным паролем: неверный пароль или файл повреждён
```

Рисунок 3.4 – Зашифрованный файл эталонной хэш-суммы и отказ при неверном пароле файла

Неверный пароль отклоняется до расшифрования, верный возвращает файл байт в байт, и проверка целостности снова сходится.

Зашифрованные файлы не содержат исходного текста; случайные соль и IV делают шифртексты одного файла различными при каждом шифровании. Скорость реализации AES – около 370 КБ/с – достаточна для текстовых файлов.

При зашифрованном журнале любая операция, кроме его расшифрования, отклоняется: на рисунке 3.5 показан отказ в добавлении записи ещё до ввода логина и пароля.

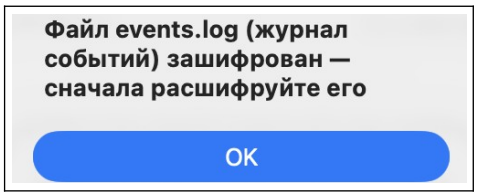


Рисунок 3.5 – Отказ в добавлении учётной записи при зашифрованном журнале

3.4 Журнал событий

Каждая строка журнала событий содержит дату, время и описание события. В журнал заносятся все операции пункта 4 задания, а также проверки паролей учётных записей и отказы при неверном пароле файла; по таким отказам видна попытка подбора пароля к БД или эталону.

Журнал после сценария «добавление – изменение – проверка паролей – целостность – внешняя правка – удаление – сохранение эталона – шифрование, неверный пароль файла, расшифрование» приведён на рисунке 3.6.

```
2026-09-12 22:00:15  создана пустая БД учётных записей, сохранена её хэш-сумма
2026-09-12 22:00:15  добавлена учётная запись «ivan»; хэш-сумма БД обновлена: 530604dc790e4a33…
2026-09-12 22:00:15  добавлена учётная запись «maria»; хэш-сумма БД обновлена: 0d0b82eece3f0349…
2026-09-12 22:00:15  добавлена учётная запись «admin»; хэш-сумма БД обновлена: 4745ac44fdc9109a…
2026-09-12 22:00:15  изменена учётная запись «maria»: пароль изменён;
                     хэш-сумма БД обновлена: b931004efa1b0536…
2026-09-12 22:00:15  проверка пароля «ivan»: верный
2026-09-12 22:00:15  проверка пароля «admin»: НЕВЕРНЫЙ
2026-09-12 22:00:15  проверка целостности БД: БД в целостности
2026-09-12 22:00:15  проверка целостности БД: БД БЫЛА ИЗМЕНЕНА ИЗВНЕ (эталон b931004efa1b0536…,
                     актуальная a15b38ffb4786160…)
2026-09-12 22:00:15  удалена учётная запись «hacker»; хэш-сумма БД обновлена: b931004efa1b0536…
2026-09-12 22:00:15  сохранена эталонная хэш-сумма БД: b931004efa1b0536…
2026-09-12 22:00:15  зашифрован файл: хэш-сумма БД (accounts.hash)
2026-09-12 22:00:15  ОТКАЗ в расшифровании файла accounts.hash: неверный пароль или файл повреждён
2026-09-12 22:00:15  расшифрован файл: хэш-сумма БД (accounts.hash)
```

Рисунок 3.6 – Содержимое файла журнала событий

Принцип «нет журнала – нет операции» проверен автотестом: если строку журнала записать не удалось, добавление записи, сохранение эталона, шифрование и расшифрование файла отменяются, и все файлы остаются прежними.

3.5 Анализ результатов

Результаты испытаний показывают, что подсистемы защиты дополняют друг друга и закрывают разные угрозы: хэширование паролей делает утечку файла БД бесполезной для входа в систему, контроль целостности обнаруживает правку БД в обход программы, шифрование скрывает содержимое файлов при хранении, а журнал позволяет восстановить ход событий и заметить попытки подбора паролей. Конфликтов между подсистемами не выявлено: после расшифрования проверка целостности сходится, эталон меняется только вместе с БД, операции с зашифрованными файлами блокируются, а при сбое записи журнала операция отменяется.

Автотест модуля хранилища повторяет сценарий во временной папке: все двадцать шесть проверок пройдены (таблица 3.1). Самопроверка модуля шифрования AES: шифртексты AES-128, AES-192 и AES-256 совпадают с векторами FIPS-197, режим CBC обратим на данных от 0 до 1000 байт.

Таблица 3.1 – Автоматическая проверка подсистем

Подсистема	Проверяемое свойство	Пройдено
Учётные записи	Пароли не хранятся открыто; верный и неверный пароль различаются; смена пароля, переименование; дубликат и логин с «#» отклонены; своя соль у каждой записи	9/9
Целостность	Совпадение после изменений через программу; внешняя правка обнаружена; эталон обновлён; испорченные файлы хэш-суммы и БД дают сообщение об ошибке	5/5
Шифрование	Три файла зашифрованы и нечитаемы; операции с ними блокируются; неверный пароль файла отвергнут; после расшифрования БД в целостности, записи сохранились	9/9
Журнал	Все виды событий присутствуют; дата и время в каждой строке; при сбое записи журнала операция отменяется	3/3

Пункты задания подтверждены: пункт 1 (см. рис. 2.1, 2.2, 3.1), пункт 2 (см. рис. 3.2), пункт 3 (см. рис. 3.3–3.5), пункт 4 (см. рис. 3.6), все подсистемы (см. таблицу 3.1).

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Почему, как правило, пароли не хранятся в открытом виде? Как обеспечивать хранение паролей не в открытом виде, при этом имея возможность проверять их правильность?

БД с открытыми паролями при утечке или инсайдерском доступе даёт вход в систему, а пользователи часто повторяют пароли на других сервисах. Хэширование решает задачу благодаря детерминированности и односторонности хэш-функции: хранится хэш от соли, соединённой с паролем, при входе хэш пересчитывается с той же солью и сравнивается – правильный пароль даёт совпадение, а восстановить пароль из хэша нельзя (см. рис. 3.1). Соль исключает одинаковые хэши и предвычисленные таблицы, медленная функция (PBKDF2, bcrypt, Argon2) замедляет перебор.

Вопрос 2. Для чего нужна подсистема аудита с точки зрения безопасности?

Аудит фиксирует, кто, когда и что сделал в системе; в программе с одним оператором журнал хранит дату, время и описание операции (см. рис. 3.6). Это позволяет обнаружить подозрительную активность (серию неверных паролей, доступ в нерабочее время), расследовать инцидент по последовательности событий, доказать факт действий и сдержать внутренних нарушителей. Журнал сам нужно защищать от правки: в работе он шифруется при хранении (см. рис. 3.5), в промышленных системах передаётся на отдельный сервер или подписывается.

Вопрос 3. Каким образом с помощью хэш-функции возможна проверка целостности цифрового объекта?

Хэш-функция отображает объект в короткое значение фиксированной длины: любое изменение входа меняет значение, а подобрать другой объект с тем же хэшем практически невозможно (стойкость ко второму прообразу). Эталонная хэш-сумма вычисляется, когда объект заведомо цел, и хранится отдельно; позже хэш пересчитывается и сравнивается: совпадение означает, что объект не изменён (см. рис. 3.2). Эталон нужно защищать, иначе его подменят вместе с объектом.

Вопрос 4. Для чего используется криптографическая подсистема безопасности?

Она обеспечивает конфиденциальность: без ключа содержимое файлов нельзя прочитать даже при полном доступе к диску или резервным копиям (см. рис. 3.4). Дополнительно она даёт аутентичность шифртекста (HMAC), а с асимметричными алгоритмами – электронную подпись и распределение ключей. Отдельные ключи для разных файлов ограничивают ущерб при компрометации одного, а вывод ключа из пароля через PBKDF2 замедляет перебор. Шифрование защищает данные в хранении и при передаче, остальные угрозы закрывают хэширование паролей, контроль целостности и аудит.

ЗАКЛЮЧЕНИЕ

Разработан менеджер учётных записей с графическим интерфейсом и четырьмя подсистемами защиты: пароли хранятся как солёные хэши PBKDF2-HMAC-SHA256; целостность БД контролируется эталонной хэш-суммой SHA-256, сохраняемой при каждом изменении и сверяемой по запросу с сообщениями «БД в целостности» и «БД была изменена извне»; файлы БД, эталона и журнала шифруются AES-256-CBC с отдельным ключом для каждого файла и кодом аутентификации HMAC; журнал фиксирует дату и время всех операций.

Пункты 1–4 задания выполнены и подтверждены экранами программы, содержимым файлов и автотестом из двадцати шести проверок; реализация AES совпадает с векторами FIPS-197. Шифрование не конфликтует с целостностью: хэш считается по открытым данным, шифрование обратимо байт в байт, операции с зашифрованными файлами блокируются.

Ограничения учебной реализации – бесключевой эталон и скорость шифра на чистом Python. SHA-256 без ключа защищает от случайных и неквалифицированных правок, а злоумышленник с правом записи может пересчитать эталон или вернуть старую пару файлов. Шифрование файла эталона не даёт подделать сам зашифрованный эталон (без пароля файла HMAC не сойдётся), но не мешает заменить его открытым файлом с пересчитанной хэш-суммой: программа распознаёт шифрование только по строке-признаку и не помнит, что файл был зашифрован; от отката защиты тоже нет. В общем случае нужны секрет, недоступный атакующему (HMAC или электронная подпись эталона, как в лабораторной работе № 4), носитель только для чтения и учёт версий эталона. В открытом виде журнал не защищён от правки и удаления, поэтому в промышленных системах его пишут на отдельный сервер или подписывают; здесь шифрование защищает его лишь при хранении, между сеансами работы.